Software Architecture Document — Online Examination Portal

Course: UE24CS341A — Software Engineering, PES University, Dept. of CSE
Project: Online Examination System with Multiple-Choice Questions and Evaluation
Team: Kshitij G Shettigar (Backend & Data) · Kartik Basavaraj Meti (Frontend & UX) · Komal S Sajja (QA & Docs)
Version: 2.0 · **Date:** 2026-11-03 (2.0 adds traceability, security architecture and the design section: sequence diagrams, API design and error handling; 1.0 was 2026-10-15)

1. Introduction

1.1 Purpose

This document records the architecture and design of the Online Examination Portal. It covers the architectural pattern and why it was chosen, the components and their interfaces, the security architecture, the request-level design (sequence diagrams, API and error handling) and the technology each layer is built with.

It follows the structure of IEEE 1016 (Software Design Descriptions): an architectural viewpoint (§2–§6) followed by an interaction and interface viewpoint (§7).

1.2 Scope

The architecture covers the 30 functional and 14 non-functional requirements of `docs/SRS.pdf` as built on `main` at the end of Sprint 2 (build `2d74453`). The section numbers §3.1 and §6 are cited in code comments, so they are kept stable across versions.

1.3 References

Document	Where
Software Requirements Specification (FR-001 … FR-030, NFR-001 … NFR-014)	`docs/SRS.pdf`
Use case diagram and flows (UC-01 … UC-13)	`docs/Use_Case_Diagram.pdf`, `docs/Use_Case_Flow.pdf`
Test Plan (119 cases)	`docs/Test_Plan.pdf`
Deviations register (every change from these documents)	`docs/Deviations.pdf`

2. Architectural pattern

We chose a Layered architecture for the Online Examination Portal.

2.1 Styles compared

Style	Fit for this system	Verdict
Layered (Presentation / Business / Data)	One deployable on one server; every rule (timer, eligibility, scoring) sits in one Business layer over one consistent database	**Chosen**
Microservices	Independent scaling is not needed at 200 concurrent attempts (NFR-001), and splitting attempts and scoring across services breaks the single transaction that submission needs (FR-020, FR-022)	Rejected — operational cost with no requirement that pays for it
Client-Server	Topologically true — browsers are the clients of one server — but says nothing about how the server is organised	Absorbed — the layered server *is* the server

2.2 Reasons for the choice

1. **One consistent record for each attempt.** Starting an attempt, saving responses, submitting and scoring all read and write the same attempt row (FR-016, FR-019, FR-020, FR-022). In one Business layer over one database this is a single database transaction; across services it would need distributed coordination.
2. **The rules live on the server, behind one layer.** The timer is computed from the server clock (FR-017), eligibility from server time and enrollment (FR-015), and the answer key never leaves the server (FR-029). Layering puts all of these in the Business layer, where the browser cannot reach or alter them.

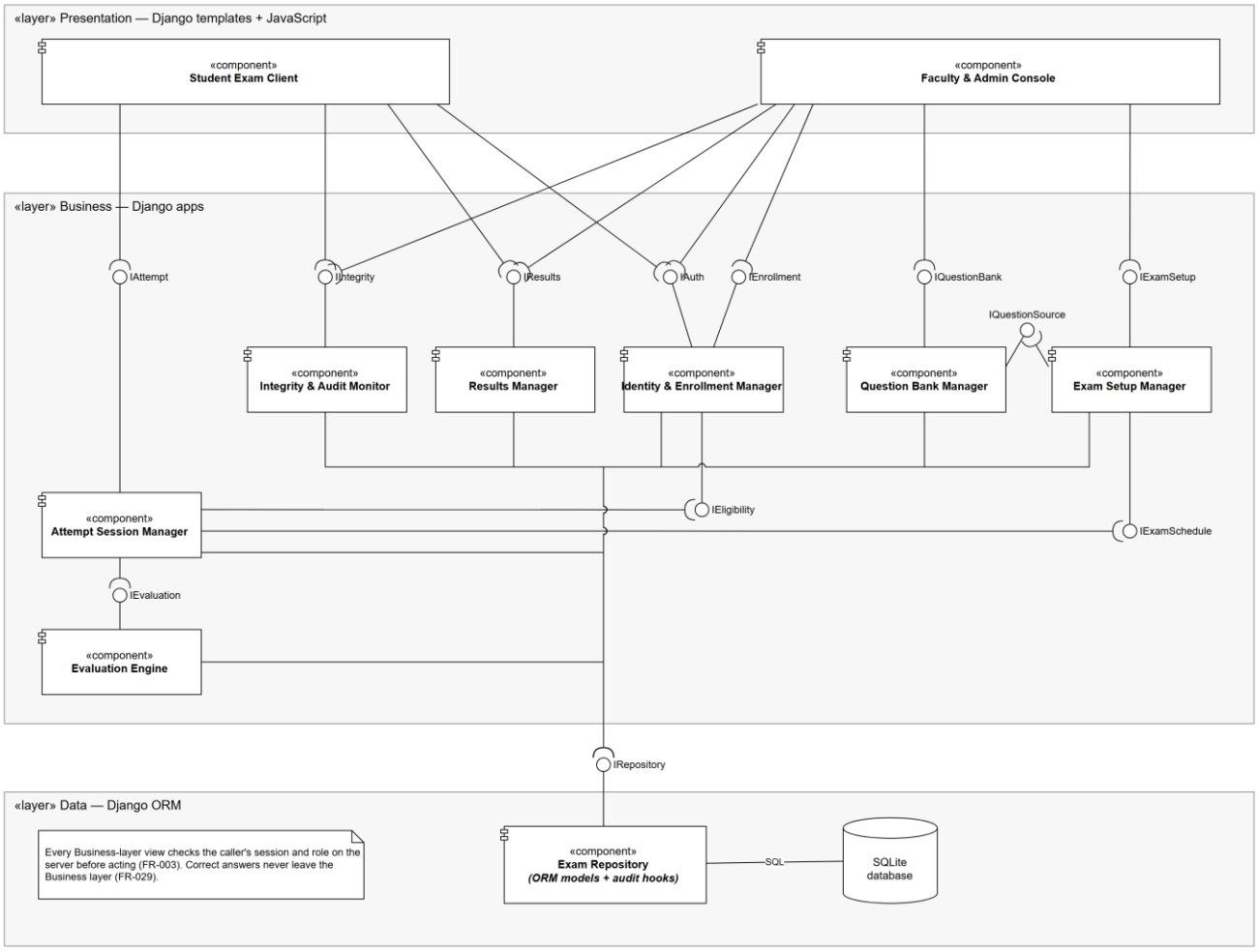
2.3 Security advantage

The Presentation layer has **no path to the database**. It can only call Business-layer interfaces, and every Business-layer view checks the caller's session and role before acting (FR-003). The Attempt Session Manager builds the question payload for the Student and leaves out the correct option; the Presentation layer has no other route to it, so the key cannot leak through the page (FR-029, verified by `IT_035`). §4 describes the security architecture in full.

2.4 Performance benefit

Evaluation runs inside the server, on data already in the same database, as soon as the attempt is submitted — no network hop between services. Scoring a 100-question attempt is one query for the responses and one bulk update of their marks, well within the 5-second target (NFR-002). Every page reads through one repository with indexed foreign keys, which is what NFR-001 (95% of requests within 2 s at 200 concurrent attempts) is load-tested against.

3. Component architecture



Notation: a ball is a **provided** interface on the component that implements it; a socket is a **required** interface on the component that calls it; a ball in a socket is an assembly connector.

3.1 Components

Layer	Component	Django app	Responsibility	Requirements
Presentation	**Student Exam Client**	templates + `attempts/static`	Dashboard, exam screen, server-driven countdown, question palette, autosave and focus-loss listeners, scorecard	FR-014, FR-017, FR-018, FR-019, FR-027, FR-028
Presentation	**Faculty & Admin Console**	templates	Course and enrollment screens, question editor, exam setup and scheduling, results publishing, integrity and audit views	FR-004, FR-005, FR-007–FR-013, FR-026
Business	**Identity & Enrollment Manager**	`identity`	Registration, sign-in, roles, account lockout, course enrollment, eligibility checks	FR-001–FR-006, FR-015, NFR-011, NFR-012
Business	**Question Bank Manager**	`questionbank`	Questions, options, tags, versioning of questions in use	FR-007, FR-008, FR-009
Business	**Exam Setup Manager**	`examsetup`	Examination details and policy, question selection, Draft/Published state, schedule window	FR-010–FR-013, NFR-010
Business	**Attempt Session Manager**	`attempts`	Start attempt, server timer, answer-free paper payload, autosave, resume, submit and auto-submit	FR-015–FR-021, FR-029
Business	**Evaluation Engine**	`evaluation`	Scores a submitted attempt: positive and negative marks, total, percentage, pass/fail	FR-022, FR-023, FR-024
Business	**Results Manager**	`results`	Withholds results until published; publish and unpublish; scorecards	FR-025, FR-026, FR-027
Business	**Integrity & Audit Monitor**	`integrity`	Records focus-loss events; shows events and the audit trail to Faculty and Administrators	FR-028, FR-030
Data	**Exam Repository**	`repository`	Django ORM models over the database; writes the append-only audit entry on every change	FR-030, NFR-013

3.2 Interfaces

Interface	Provided by	Required by	Kind
`IAuth`	Identity & Enrollment Mgr	Student Exam Client, Faculty & Admin Console	HTTP view
`IEnrollment`	Identity & Enrollment Mgr	Faculty & Admin Console	HTTP view
`IQuestionBank`	Question Bank Mgr	Faculty & Admin Console	HTTP view + JSON endpoint
`IExamSetup`	Exam Setup Mgr	Faculty & Admin Console	HTTP view
`IAttempt`	Attempt Session Mgr	Student Exam Client	HTTP view + JSON endpoints (autosave, timer)
`IIntegrity`	Integrity & Audit Monitor	Student Exam Client (report events), Faculty & Admin Console (view)	HTTP view + JSON endpoint
`IResults`	Results Mgr	Student Exam Client, Faculty & Admin Console	HTTP view + JSON endpoint
`IEligibility`	Identity & Enrollment Mgr	Attempt Session Mgr	In-process call
`IExamSchedule`	Exam Setup Mgr	Attempt Session Mgr	In-process call
`IQuestionSource`	Question Bank Mgr	Exam Setup Mgr	In-process call
`IEvaluation`	Evaluation Engine	Attempt Session Mgr	In-process call, same transaction as submission
`IRepository`	Exam Repository	all seven Business components	Django ORM

Count: 7 Presentation→Business, 4 Business→Business, 1 Business→Data. Nothing in the Presentation layer requires `IRepository`. §7.2 lists the HTTP endpoints behind each HTTP interface.

3.3 Traceability

Each component, the requirements it realises and the Test Plan cases that verify them. Every FR maps to exactly one owning Business component; `portal/traceability.py` holds the case-to-FR matrix in code and a test fails if any case loses its automated test (NFR-007).

Component	Requirements	Test Plan cases
Identity & Enrollment Manager	FR-001–FR-006, NFR-011, NFR-012	UT_001–UT_021, IT_001–IT_011, IT_038, ST_001–ST_004
Question Bank Manager	FR-007, FR-008, FR-009	UT_022–UT_027, IT_012–IT_014, ST_005
Exam Setup Manager	FR-010–FR-013, NFR-010	UT_028–UT_035, IT_015–IT_017, ST_006, ST_007
Attempt Session Manager	FR-014–FR-021, FR-029	UT_036–UT_052, IT_018–IT_028, IT_035, ST_007–ST_010
Evaluation Engine	FR-022, FR-023, FR-024, NFR-002	UT_053–UT_058, IT_028–IT_031, ST_011
Results Manager	FR-025, FR-026, FR-027	UT_059–UT_063, IT_032–IT_034, ST_012
Integrity & Audit Monitor	FR-028, FR-030	UT_064–UT_068, IT_036, IT_037, ST_013
Exam Repository	FR-030, NFR-004, NFR-013	IT_036, IT_037; `portal/test_restart.py`, `portal/test_security.py`
Student Exam Client	FR-017, FR-018, FR-028, NFR-005, NFR-009	UT_042–UT_044, IT_021, IT_022, UT_064–UT_067; `portal/test_responsive.py`, `portal/test_accessibility.py`

4. Security architecture

4.1 Trust boundaries

There are two boundaries. The **browser** is untrusted: everything it sends is validated on the server, and nothing it reports (its clock, which option is correct) is believed. The **database** is reached only through the Django ORM in the Data layer. Business components never build SQL.

4.2 Controls, layer by layer

Threat	Control	Where it lives	Requirement
Using another role's screens or endpoints	`RoleMiddleware` denies by default: a view without a `@roles(...)` declaration is refused to everyone, so a forgotten decorator fails closed	`identity/access.py`, every request	FR-003
Reading another Student's attempt or result	Every attempt and result query filters on `student=request.user`, so another Student's id returns 404 or 403	`attempts/views.py`, `results/views.py`	FR-006, FR-025
Leaking the answer key to the browser	The paper payload selects only option `id` and `text`; `is_correct` never leaves the Business layer	`attempts.views.paper`	FR-029
Extending time with the client clock	Remaining time = server `started_at` + duration − server now; the browser only displays it	`attempts.services.expires_at`	FR-017, FR-020
Password theft from a database leak	Django's salted PBKDF2-SHA256 hashing; passwords are never logged	Django auth, `repository.User`	NFR-011
Password guessing	5 failed sign-ins in 15 minutes lock the account for 15 minutes, checked before the password	`identity/lockout.py`, `identity/forms.py`	NFR-012
Cross-site request forgery	`CsrfViewMiddleware` on every POST; the page scripts post the form's CSRF token with each autosave and focus-loss report	`portal/settings.py`	NFR-013
SQL injection	ORM only: a code search finds no `.raw()`, `cursor()` or `.extra()` outside tests	all apps	NFR-013
Information leak in errors	DEBUG is off unless `DJANGO_DEBUG=1`; custom	`portal/error_handlers`	NFR-013

Threat	Control	Where it lives	Requirement
	400/403/404/500 handlers render one generic page	`.py`	
Secrets in the public repository	`SECRET_KEY` is read from `DJANGO_SECRET_KEY`; the app refuses to start without it	`portal/settings.py`	NFR-014
Clickjacking	`XFrameOptionsMiddleware` sends `X-Frame-Options: DENY`	`portal/settings.py`	NFR-013
Untraceable changes	Every create, update and delete of a user, question, exam, answer key or result writes an `AuditEntry`; no view edits or deletes one	`repository/audit.py`	FR-030

4.3 Where security is validated

The Test Plan's §5.1 maps each control above to the test cases and manual checks that verify it. The System Validation Report records the results of the manual checks.

5. Technology stack and deployment

5.1 Technology stack

Layer	Technology	Why
Presentation	Django templates, HTML/CSS, plain JavaScript (timer display, autosave on every change, `visibilitychange` and `blur` listeners)	Server-rendered pages keep the answer key server-side by default; no separate front-end build
Business	Django 5 (Python 3.12) — one Django app per Business component	Built-in authentication, password hashing (NFR-011), CSRF protection and ORM (NFR-013)
Data	Django ORM over SQLite in WAL mode	Relational data — enrollment, attempts and scores — in one file database; the ORM keeps a later move to PostgreSQL a settings change
Automation	GitHub Actions — runs `python manage.py test` on every push and pull request	Course requirement (Sprint 1); a failing test blocks the merge

Auto-submission without a background worker. An expired attempt is submitted and evaluated the first time any request touches it: every attempt view, the autosave, timer and focus-loss endpoints call `close_if_expired()` first, which stamps the submission at the exact expiry time (FR-020, FR-022). §6 records the limit this leaves. The open attempt page also reloads itself at zero, so a Student who is still on the page sees the submission at once. This avoids a task queue the system does not otherwise need.

Correction in 2.0: version 1.0 also described a `close_expired_attempts` command run by cron. It was never built, because the request-time check covers every attempt that is looked at again (recorded in `docs/Deviations.pdf`, D-13).

5.2 Deployment

One Linux virtual machine runs the Django application under Gunicorn behind Nginx (HTTPS), with the SQLite database file on the same machine and its clock synchronised by NTP. Secrets such as `SECRET_KEY` come from environment variables, never from the repository (NFR-014). After a restart the service is back within 60 seconds and every attempt resumes from its stored start time and saved responses (NFR-004).

6. Known limits

Limit	Accepted because	Upgrade path
Single server — a single point of failure	The requirement is recovery (NFR-004), not zero downtime; autosave and server-held start times make every attempt resumable	Run two app servers behind the load balancer once the database moves to PostgreSQL
SQLite allows one writer	200 students autosaving is about 40 short writes per	Switch the DATABASES setting to

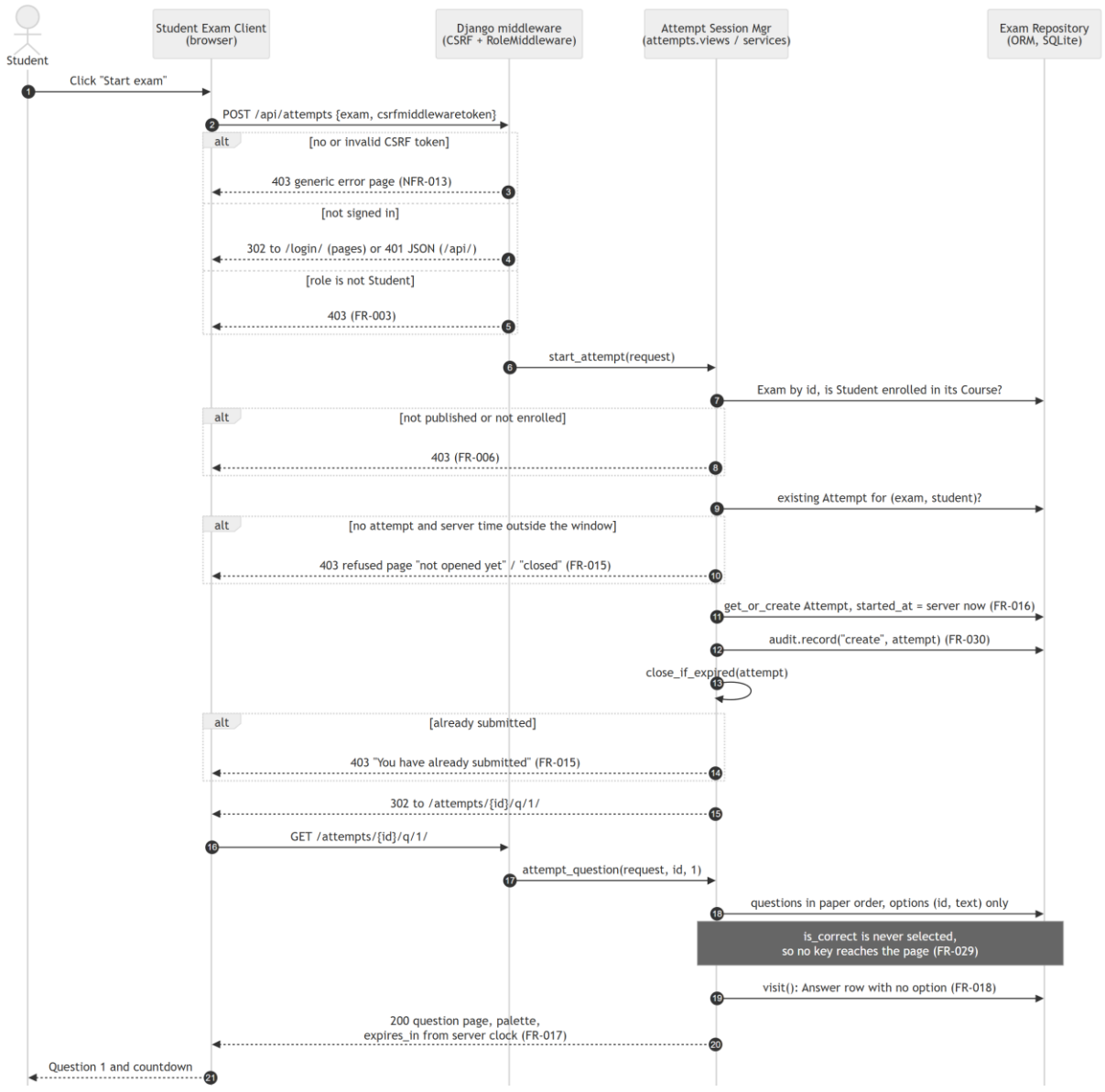
Limit	Accepted because	Upgrade path
at a time	second, which SQLite in WAL mode with `IMMEDIATE` transactions handles; NFR-001's load test confirms it	PostgreSQL; no code changes
An expired attempt whose Student never comes back is submitted only when a request next touches it	The Student's pages, autosave, the timer and focus reporting all call `close_if_expired()` first, so the Student can never act on it; it is stamped at its true expiry whenever it closes. Until then the Faculty views list it as not submitted	A cron-run sweep calling `close_if_expired()` on every open attempt past its expiry

7. Design

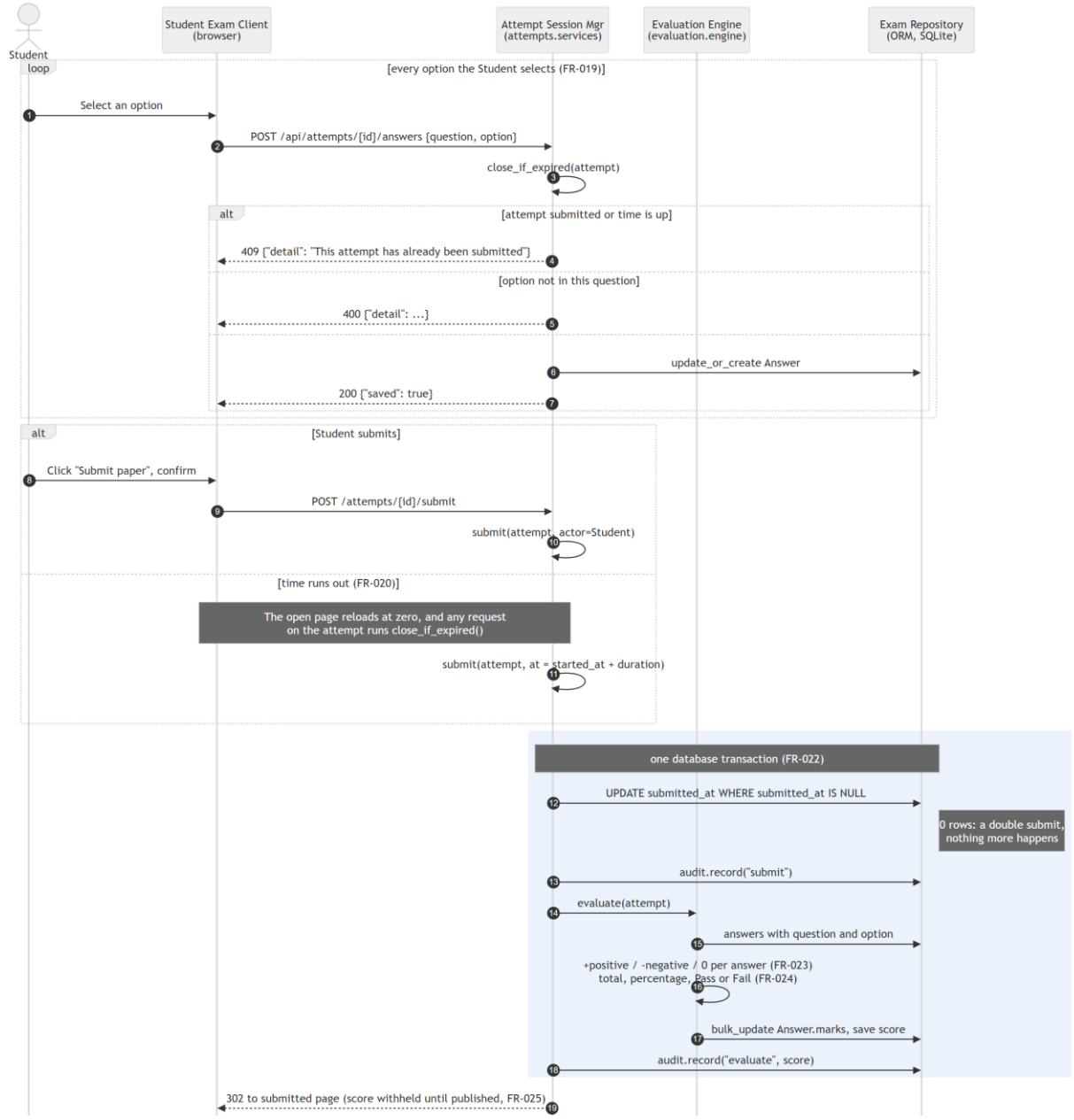
7.1 Sequence diagrams

Three interactions carry the system's core rules. Each diagram follows the code on `main`; numbers on the arrows are message order, and `alt` frames show the refusals.

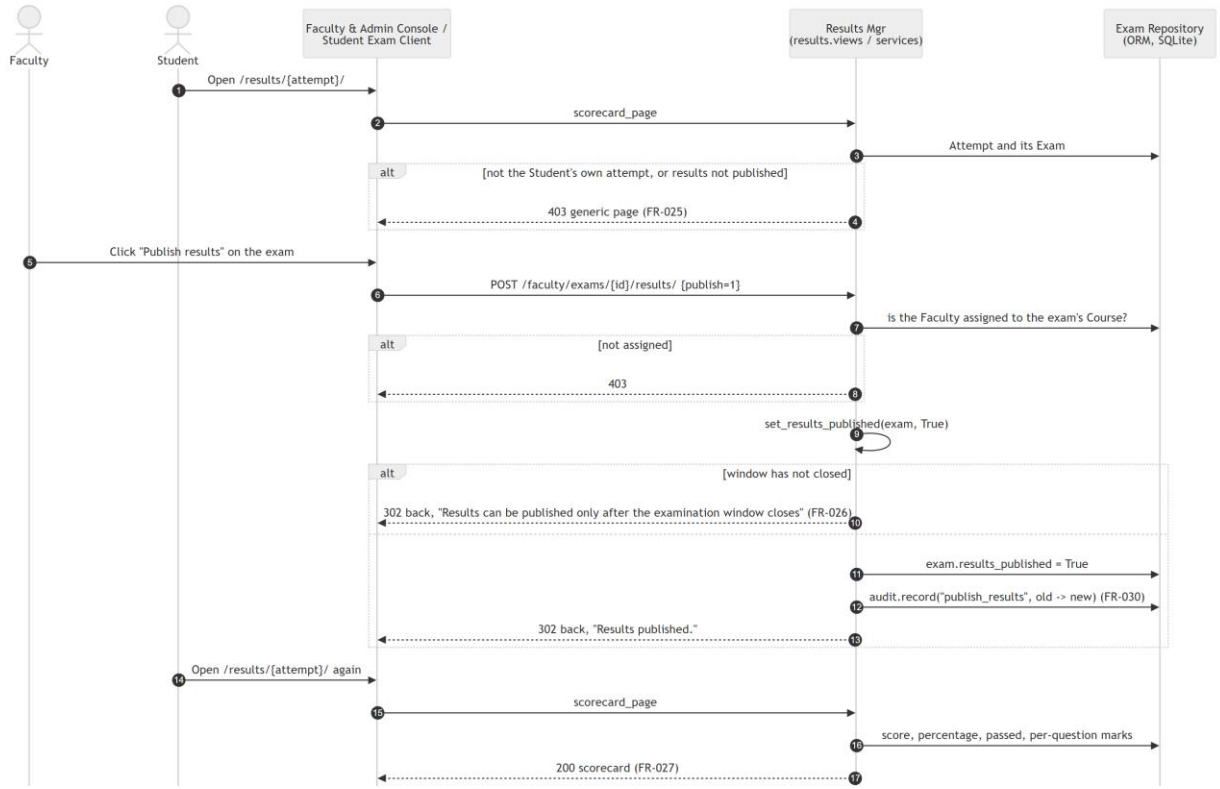
Sequence 1 — Start an attempt and receive the paper (UC-08: FR-003, FR-006, FR-015, FR-016, FR-017, FR-018, FR-029). The attempt row and its server start time exist before any question is sent, and the question payload is built without the answer key.



Sequence 2 — Autosave, submission and automatic evaluation (UC-09, UC-10, UC-11: FR-019, FR-020, FR-022, FR-023, FR-024, FR-030). Manual submission and the time-out share one `submit()`. It marks the attempt submitted and evaluates it in one transaction, so a submitted attempt is never left without a score, and a second submit changes nothing.



Sequence 3 — Publish results and view the scorecard (UC-12: FR-025, FR-026, FR-027, FR-030). A score stays hidden until the course's Faculty publishes the results, which is allowed only after the window has closed.



7.2 API design

The portal is server-rendered, so most endpoints return HTML pages and form posts redirect (Post/Redirect/Get). The JSON endpoints under `/api/` serve the in-page scripts: autosave, the timer and focus-loss reporting. Every endpoint needs a signed-in user of the listed role (FR-003), and every POST needs the CSRF token (NFR-013).

Conventions.

- Resources are addressed by integer id in the path (`/attempts/{id}/`).
- JSON responses carry either the result or `{"detail": "<message>"}`.
- Form posts answer `302` on success and re-render the form with inline errors on invalid input.
- An attempt id that is not the caller's own answers `404`, so the endpoint does not reveal that it exists. An exam, course or result the caller is not allowed to see answers `403`.

JSON endpoints

Method & path	Role	Request	Success	Errors	Requirement
POST `/api/attempts`	Student	form `exam=<id>`	`302` to the paper	`403` not enrolled, outside the window, already submitted · `404` no such exam	FR-015, FR-016
POST `/api/attempts/{id}/answers`	Student	form `question`, `option` (empty clears), optional `marked_for_review` `0/1`	`200 {"saved": true}`	`400` malformed or option not in the question · `409` attempt already submitted or out of time	FR-018, FR-019
GET `/api/attempts/{id}/remaining`	Student	—	`200 {"remaining_seconds": n}` (server clock; an expired attempt is submitted first)	`404` not the caller's attempt	FR-017, FR-020
POST `/api/attempts/{id}/focus-lost`	Student	no body	`204`	`409` attempt already submitted · `404` not the caller's attempt	FR-028

Method & path	Role	Request	Success	Errors	Requirement
GET /api/results/{attempt}	Student	—	200 {"score", "percentage", "passed"}	403 another Student's attempt or results not published	FR-025
POST /api/questions	Faculty	JSON question with options	201 {"id": n}	400 {"errors": {field: [messages]}} invalid JSON or fields	FR-007, FR-008

Unauthenticated calls to /api/ get 401 {"detail": "Authentication required."} rather than a redirect, because a script cannot follow a sign-in page.

Page endpoints

Area	Paths	Role	Requirement
Accounts	GET/POST /register/, /login/, POST /logout/	public	FR-001, FR-002, NFR-008, NFR-012
Dashboards	/student/, /faculty/, /admin/	own role	FR-002, FR-014
Courses and enrollment	/admin/courses/ (+ new/, {id}/edit/, {id}/archive/), /faculty/courses/, /courses/{id}/roster/, /courses/{id}/enroll/, /courses/{id}/unenroll/{student}/	Admin, Faculty	FR-004, FR-005
Data erasure	POST /admin/students/{id}/erase/	Admin	NFR-008
Question bank	/faculty/questions/ (+ new/, {id}/edit/, {id}/delete/)	Faculty	FR-007–FR-009
Examinations	/faculty/exams/ (+ new/, {id}/, {id}/edit/, {id}/questions/, {id}/publish/, {id}/schedule/)	Faculty	FR-010–FR-013
Attempt	/student/exams/, /student/exams/{id}/, /attempts/{id}/, /attempts/{id}/q/{n}/, POST /attempts/{id}/submit	Student	FR-006, FR-014–FR-021
Results	POST /faculty/exams/{id}/results/ (publish=1 or 0), /results/{attempt}/	Faculty, Student	FR-025–FR-027
Integrity and audit	/faculty/exams/{id}/integrity/, /admin/audit/?entity=&entity_id=	Faculty, Admin	FR-028, FR-030

7.3 Error handling protocols

Status codes. The same failure gets the same answer everywhere:

Situation	Response	Example
Not signed in	Page: 302 to /login/?next=… · API: 401 JSON	Opening /student/ after sign-out
Signed in, wrong role or someone else's resource	403 generic page (API: 403)	A Student opening /faculty/questions/
Missing CSRF token	403	A forged POST from another site
No such resource	404 generic page	/attempts/9999/
Malformed input	Form: re-rendered with inline field errors · API: 400 with detail or errors	Question with seven options
Valid request, wrong state	API: 409 with detail · Form: refusal page or message	Autosave after the time is up
Business rule refusal at start	403 refusal page naming the reason	"This examination has not opened yet"
Unexpected exception	500 generic page; the transaction in progress is rolled back	—

Rules.

3. **Fail closed.** A view that declares no roles is refused; an unknown attempt state is treated as closed; a request to save into a submitted attempt is refused, never applied.
4. **No internals in responses.** DEBUG is off, and the four handlers in portal/error_handlers.py render one generic errors/error.html page with no stack trace or SQL (NFR-013).

5. **State changes are atomic.** Submission and evaluation run in one transaction, so a crash leaves the attempt either open with its saved answers or submitted with a score, never half-done (FR-022, NFR-004). The conditional `UPDATE … WHERE submitted_at IS NULL` makes a double submit, or a manual submit racing the time-out, a no-op.
6. **Saves are idempotent.** Autosave is an upsert keyed on (attempt, question), so sending the same response twice leaves one row. A `409` makes the page reload, which shows the submitted page. The script does not retry a save that failed on the network; the palette cell only turns Answered once the server confirms the save, so the Student can see what is stored.
7. **Database contention waits instead of failing.** SQLite runs in WAL mode with `IMMEDIATE` transactions and a 20-second busy timeout, so concurrent autosaves queue rather than raise "database is locked" (NFR-001).
8. **Messages say what to do.** A refusal names the reason in the Student's terms (not opened yet, closed, already submitted); form errors sit next to the field that caused them.